

GUIDE DE CONCEPTION

Agentic OS

Comprendre, concevoir et exploiter
un système d'agents fiable

Architecture • Mémoire • Orchestration
Sécurité • Évaluation • Mise en production

Recherche du 30 septembre 2026
Version 1.0 • Français

À partir de 9 sources initiales et d'un registre de 35 références.
Avec une proposition de réalisation Symfony et un cas pratique de recherche sourcée.

Ce guide part de neuf liens fournis par Nicolas Rodriguez. Il confronte leurs propositions à des documentations officielles, des travaux de recherche et des référentiels de sécurité. Son objectif est de permettre de construire un système utile, vérifiable et maintenable, avec une première déclinaison adaptée à un développeur Symfony.

Résultat principal. Dans le corpus étudié, « Agentic OS » désigne plusieurs architectures et positionnements commerciaux. Le terme ne suffit donc pas à décrire un produit. Pour ce guide, il désigne une couche applicative qui organise des tâches confiées à des agents, leur contexte, leurs outils, leurs permissions et leur suivi dans le temps. Il ne remplace pas Windows ou Linux.

Mode de lecture. Les références [S01] à [S35] renvoient au registre final. Les paragraphes marqués « Proposition » sont des recommandations de conception élaborées pour ce guide, pas des capacités certifiées d'un logiciel. Les exemples et seuils sont illustratifs. Aucun des produits cités n'a été installé ni testé dans cette recherche.

Sommaire

1. Périmètre et méthode de recherche
2. Ce que les neuf sources apportent réellement
3. Définition et vocabulaire
4. Affirmations à corriger ou à nuancer
5. Architecture de référence
6. Orchestration et cycle d'une tâche
7. Outils, skills et protocoles
8. Mémoire, contexte et connaissances
9. Gouvernance et niveaux d'autonomie
10. Sécurité des agents
11. Fiabilité, reprises et observabilité
12. Évaluer avant d'automatiser
13. Choisir les modèles et maîtriser le coût
14. Choisir une pile technique
15. Une réalisation possible avec Symfony
16. Cas pratique : un dossier de recherche sourcé
17. Déploiement et exploitation
18. Feuille de route et critères d'acceptation
19. Cahier des charges de départ
20. Registre des sources et limites

1. Périmètre et méthode de recherche

Les neuf pages initiales ont été ouvertes. Leurs liens utiles ont été suivis de manière sélective : documentation Claude Code, documentation StackGen, article ReAct, mémoire Obsidian, gouvernance Acumentica et outils de sécurité cités par EM360Tech. Des liens de deuxième niveau ont ensuite conduit au standard Agent Skills, à l'article d'architecture Managed Agents et au PDF OWASP consacré aux applications agentiques. Des sources officielles complémentaires ont servi au recoupement.

Il s'agit d'une **collecte documentaire ciblée**, et non d'un crawl récursif exhaustif de tous les sites. Les menus, publicités, réseaux sociaux, pages d'inscription et contenus sans rapport direct ont été écartés. Les vidéos intégrées n'ont pas été transcrites. Les contenus réservés aux membres n'ont pas été consultés. Les pages longues ont été examinées sur leurs passages pertinents ; les tests et benchmarks qu'elles mentionnent n'ont pas été reproduits.

La date de consultation n'est pas la date de publication. Les pages web peuvent être modifiées sans changer leur URL. Une page commerciale est une source primaire pour savoir ce que son éditeur annonce ; elle ne constitue pas une preuve indépendante de performance.

Hierarchie des preuves

Niveau	Type de source	Usage dans le guide
A	Spécification, documentation officielle, dépôt officiel	Décrire un mécanisme ou une interface documentée
B	Article de recherche ou retour technique de l'auteur	Expliquer un choix ou un résultat dans son périmètre
C	Page produit ou article commercial	Identifier une offre, une idée ou une promesse
D	Classement et synthèse éditoriale	Découvrir des pistes à confirmer auprès des auteurs

La convergence de plusieurs pages commerciales ne suffit pas à valider une affirmation. Deux articles du même éditeur restent une seule famille de sources. Une référence circulaire entre pages ne remplace pas une spécification, un protocole d'essai ou un résultat mesuré.

2. Ce que les neuf sources apportent réellement

Source de départ	Apport retenu	Limite de preuve
MindStudio : skills et contexte [S01]	Contexte partagé, versionnement, retour d'expérience, synchronisation	Assimile trop directement les skills à des fonctions typées
Nirmata.in [S02]	Vision d'une fonction métier opérée en continu	Promesses d'autonomie, d'amélioration et de ROI non établies par les éléments consultés
Acumentica : entreprise OS [S03]	Mandat, incertitude et contrôle des décisions	Cadre conceptuel commercial ; peu de détails vérifiables d'implémentation
StackGen [S04]	Plateforme spécialisée DevOps/SRE ; identité, budgets, politiques, audit annoncés	Capacités décrites par l'éditeur ; documentation cible non extraite
SDLC Corp [S05]	Introduction aux plans, outils, mémoire et défaillances	Source pédagogique de prestataire, à recouper avec les références originales
EM360Tech [S06]	Panorama des familles de risques et d'outils	Classement éditorial mêlant logiciels, services et référentiels
Acumentica : Control OS [S07]	Bornage des boucles et gouvernance transversale	Ne démontre pas une garantie d'absence de dérive
AI Profit Boardroom [S08]	Interface personnelle, workflows et mémoire Markdown	Tunnel commercial ; plusieurs raccourcis sur gratuité et sécurité
MindStudio : AI OS [S09]	Inventaire des composants d'une plateforme	Un inventaire utile, pas une architecture universelle imposée

Le corpus couvre trois usages : **assistant personnel, automatisation métier, plateforme opérationnelle d'entreprise**. Leurs besoins diffèrent. Un assistant de recherche personnel peut commencer avec des fichiers et un seul agent. Une plateforme qui modifie des systèmes de production doit gérer des identités, des validations, la concurrence et les pannes.

3. Définition et vocabulaire

Anthropic distingue les workflows dont le parcours est défini dans le code des agents qui choisissent dynamiquement leurs étapes et outils. Cette distinction est plus utile pour concevoir un système que le seul mot « autonome ». [S10]

Terme	Sens pratique retenu
LLM	Modèle qui produit des sorties à partir d'un contexte
Agent	Boucle qui demande au modèle une prochaine action et exploite son résultat
Outil	Fonction exécutable : recherche, lecture, calcul ou modification
Skill	Procédure réutilisable et ressources qui guident l'agent

Terme	Sens pratique retenu
Workflow	Organisation des étapes, conditions et transitions
Harness	Logiciel qui exécute la boucle, prépare le contexte et gère les outils
Orchestrateur	Composant qui répartit et suit le travail ; il peut être déterministe
RAG	Recherche de documents ajoutés au contexte avant une génération
Plan de contrôle	Règles, identités, budgets, décisions d'autorisation et suivi
Plan d'exécution	Workers et outils qui réalisent les opérations autorisées

Un **prompt** peut décrire une règle ; il ne remplace pas le code qui la fait respecter. Une base vectorielle aide à retrouver des passages ; elle ne constitue pas un registre transactionnel. Un tableau de bord permet d'interagir avec le système ; sa présence ne prouve ni persistance ni sûreté.

Proposition : réserver l'appellation « Agentic OS opérationnel » à un ensemble qui sait recevoir un travail, le borner, l'exécuter, conserver son état, rendre son résultat vérifiable et s'arrêter proprement. Le nombre d'agents, le fonctionnement permanent et l'apprentissage automatique ne sont pas des prérequis universels.

4. Affirmations à corriger ou à nuancer

« La mémoire fait automatiquement progresser le système »

Nirmata présente mémoire et feedback comme moteurs d'amélioration continue. Cette progression n'est pas démontrée par la seule présence d'un stockage. Une mémoire peut également conserver une erreur. Proposition : enregistrer une correction, la vérifier, la transformer en changement versionné, puis mesurer son effet sur des cas réservés à l'évaluation. Cela modifie le système ; ce n'est pas nécessairement un réentraînement du modèle. [S02, S17, S25]

« Une skill Claude Code est une fonction typée »

La documentation officielle décrit une skill fondée sur un fichier d'instructions SKILL.md, avec des ressources complémentaires possibles. Les contrats exécutables et la validation des arguments relèvent des outils et du runtime. On peut construire une procédure autour d'outils typés, sans que la procédure textuelle fournisse elle-même cette garantie. [S01, S11, S12]

« Les outils d'automatisation ne peuvent pas intégrer de raisonnement »

L'opposition absolue entre workflow et agent est trompeuse. La documentation de n8n décrit un nœud AI Agent utilisant des outils. Un workflow peut donc inclure une étape agentique. Le choix pertinent porte sur les décisions laissées au modèle, pas sur une frontière commerciale entre catégories. [S02, S30]

« Obsidian est open source et suffit comme cerveau partagé »

AI Profit Boardroom qualifie Obsidian d'open source. L'application est propriétaire ; l'usage gratuit et les fichiers Markdown portables sont des propriétés différentes. Ses conditions présentent un usage gratuit et réservent les droits sur le code de l'application. Un dossier partagé reste à compléter par des contrôles d'accès et de concurrence. [S08, S20, S21]

« Local signifie sécurisé et gratuit »

Une interface locale réduit certaines expositions réseau. Elle ne protège pas automatiquement contre un fichier malveillant, une extension compromise ou un agent doté de trop de droits. Les recommandations MCP traitent explicitement la compromission des serveurs locaux. Le calcul local consomme du matériel et de l'énergie ; une API distante peut recevoir les données même si l'interface est locale. [S08, S15]

« Gouvernance signifie absence de dérive »

Acumentica formule des promesses de stabilité et de conformité aux mandats. Le guide retient le besoin de contrôle, sans reprendre ces promesses comme garanties. Proposition : traduire chaque exigence en autorisation vérifiable, test et preuve d'exécution. Aucun label « OS » ne suffit à assurer cela. [S03, S07, S22]

« ROI positif en 60 à 90 jours »

Cette affirmation apparaît chez Nirmata. Les éléments consultés ne fournissent pas de protocole indépendant suffisant pour la généraliser. Mesurer son propre temps économisé, son coût d'exploitation et le coût des erreurs avant toute projection commerciale. [S02]

5. Architecture de référence

Proposition : construire un noyau d'exécution gouverné, puis y raccorder les capacités métier. Le modèle propose des opérations ; le logiciel contrôle si elles sont autorisées et consigne ce qui a effectivement eu lieu.

Couche	Responsabilité	Preuve attendue
Entrée	Interface, webhook, planification, identité de l'émetteur	Événement daté et authentifié
Tâches	Objectif, étapes, dépendances, échéance	État durable et transitions valides
Contexte	Sélection des informations utiles et autorisées	Références et versions utilisées
Décision	Choix d'une action ou d'une réponse	Proposition structurée
Contrôle	Droits, périmètre, budget, approbation	Autorisation ou refus motivé
Exécution	Appel d'outil dans un environnement limité	Reçu d'exécution et résultat
Vérification	Contrôles métier et qualité	Résultat vérifié ou escalade
Exploitation	Journaux, alertes, coûts, reprise	Historique consultable et arrêt possible

Une source technique particulièrement utile est l'architecture Managed Agents d'Anthropic : elle sépare la session durable, le harness et l'environnement d'exécution. Cette séparation permet de remplacer ou de redémarrer les composants sans confondre leurs responsabilités. [S13]

Pour un premier système, ces couches peuvent vivre dans une seule application. Une séparation logique et des interfaces claires suffisent souvent avant de justifier des microservices. La frontière de sécurité doit néanmoins être réelle : un exécuter de code non fiable ne doit pas accéder au coffre des secrets du contrôleur.

Ce qui constitue la source de vérité

Proposition : la base métier fait autorité sur les tâches, les autorisations et les actions réalisées. Les fichiers documentaires font autorité sur les procédures approuvées. Les index de recherche sont reconstruisibles. La conversation du modèle ne fait pas autorité sur un paiement effectué, un courriel envoyé ou une mise à jour terminée.

6. Orchestration et cycle d’une tâche

Choisir le degré de liberté

Structure	Quand l’utiliser	Coût ou risque principal
Appel unique avec validation	Extraction ou classement simple	Résultat plausible mais incorrect
Workflow à étapes fixes	Processus métier connu	Mauvaise gestion des exceptions
Agent avec outils bornés	Recherche dont les étapes varient	Boucles inutiles et erreurs cumulées
Superviseur et spécialistes	Sous-tâches réellement distinctes	Coordination, contexte perdu, coût

Les schémas de routage, parallélisation et délégation sont documentés par Anthropic. Sa recommandation est de n’ajouter de complexité que lorsqu’elle apporte un bénéfice mesurable. [S10] Le travail ReAct illustre l’alternance entre réflexion et action, avec retour d’information de l’environnement ; il ne démontre pas à lui seul la fiabilité d’un système métier complet. [S31]

Machine d’états proposée

Une tâche passe par : reçue, prête, en cours, éventuellement en attente de validation, puis réussie, échouée ou annulée. Un délai dépassé peut provoquer une mise en échec ou une reprise selon une politique explicite. « En attente de validation » libère le worker ; l’attente est conservée en base.

Avant chaque opération : vérifier l’annulation, la limite d’étapes, la durée, le budget restant et la fraîcheur des données nécessaires. Après l’opération : enregistrer un résultat vérifiable, puis décider si l’objectif est atteint. Une phrase du modèle annonçant la réussite ne suffit pas.

Communication entre agents

Proposition : transmettre des messages structurés contenant task_id, parent_task_id, émetteur authentifié, objectif, références d’entrée, résultat attendu, budget, échéance et profondeur de délégation restante. Le résultat contient un statut, des faits sourcés, des incertitudes et des références d’artefacts.

Le destinataire vérifie lui-même ses permissions. Un agent ne peut pas augmenter les droits d’un autre en écrivant « autorisé » dans son message. Les sous-tâches consomment le budget global, et leurs budgets réservés sont comptabilisés atomiquement pour éviter un dépassement par exécutions simultanées.

7. Outils, skills et protocoles

Construire des outils étroits

Proposition : préférer « créer un brouillon pour ce dossier » à « effectuer n’importe quelle requête HTTP ». Définir pour chaque outil un schéma d’entrée, un schéma de sortie, une identité, des ressources autorisées, des délais, des erreurs explicites et le caractère réversible ou non de l’action.

Exemple de contrat applicatif illustratif :

```
{
  "name": "create_research_draft",
  "version": "1.0",
  "input": {"task_id": "uuid", "source_ids": ["S01"]},
  "output": {"draft_id": "uuid", "status": "created"},
  "side_effect": "internal_draft",
  "permission": "research.draft.create",
  "idempotency_required": true,
  "timeout_seconds": 30
}
```

Ce JSON décrit un contrat proposé, pas un format MCP ni un manifeste natif Claude Code. La validation réelle doit être implémentée côté serveur. Le tenant et les permissions proviennent de l’identité authentifiée, jamais d’une simple valeur choisie par le modèle.

Ce qu’une skill doit enseigner

La skill fournit la méthode : quand intervenir, quelles sources utiliser, dans quel ordre travailler, quand s’arrêter, comment signaler un manque et quelle forme de résultat produire. Le standard Agent Skills formalise une organisation de ces instructions et ressources. Il ne rend pas un script joint automatiquement digne de confiance. [S12]

Proposition : versionner chaque procédure, lui attribuer un propriétaire et lui associer quelques cas représentatifs. Une procédure modifiée doit repasser les contrôles de qualité avant de remplacer la version utilisée en production.

Distinguer les protocoles

Mécanisme	Rôle	Ce qu’il ne fournit pas seul
API métier	Accès contractuel aux fonctions d’un service	Une stratégie d’agent
MCP	Découverte et accès aux outils, ressources et prompts	La sûreté de tous les serveurs connectés
A2A	Interopérabilité entre services d’agents et suivi de tâches	Une confiance automatique entre organisations
Agent Skills	Distribution de procédures et ressources	Un moteur transactionnel

MCP décrit une architecture hôte/client/serveur et des primitives d’intégration. A2A décrit notamment des cartes d’agents, capacités et interactions de tâches. Les versions exactes doivent être fixées et testées lors de l’intégration. [S14, S16] Une application monolithique n’a pas besoin d’ajouter A2A pour faire communiquer deux fonctions internes.

8. Mémoire, contexte et connaissances

Séparer ce que l'on stocke

Information	Stockage proposé	Règle de confiance
Objectif et état de tâche	Base relationnelle	Autorité applicative
Notes de travail temporaires	État du run, durée limitée	Données intermédiaires
Préférences validées	Fiches versionnées	Auteur et date de validation
Documents métier	Fichiers et métadonnées	Source, droits et fraîcheur
Index lexical ou vectoriel	Index dérivé	Toujours revenir à la source
Retour d'expérience proposé	File de révision	Non actif avant validation

LangGraph distingue les checkpoints d'un fil d'exécution et les stores qui conservent des informations entre plusieurs fils. Cette distinction aide à éviter de confondre reprise d'une tâche et mémoire à long terme. [S17]

Constituer le contexte utile

Proposition : charger l'objectif courant, les règles pertinentes, les outils accessibles, l'état récent et quelques documents ciblés. Éviter de joindre tout l'historique à chaque appel. L'ingénierie du contexte traite précisément la sélection et la gestion de ces informations dans une fenêtre limitée. [S24]

Une fiche de connaissance devrait contenir : identifiant, contenu, source, auteur, date de vérification, périmètre, niveau de sensibilité, état proposé/validé/obsolète et références remplacées. Un résumé doit conserver les identifiants des pièces qui permettent de retrouver les détails.

RAG et contrôle d'accès

Proposition : filtrer les documents selon les permissions avant la recherche, puis revérifier l'accès avant de transmettre les passages au modèle. Ajouter source, date et emplacement aux extraits. Tester séparément la qualité de récupération et la fidélité de la réponse. Un document bien retrouvé peut être périmé ; une réponse bien écrite peut déformer un document correct.

Commencer par des fichiers bien rangés et une recherche textuelle. Ajouter des embeddings si l'évaluation montre un manque de rappel sémantique. Ajouter un graphe si les relations explicites entre personnes, objets ou événements sont nécessaires. Un graphe visuel de notes ne garantit pas une résolution fiable des relations métier.

Synchronisation et amélioration

MindStudio propose un mécanisme périodique de synchronisation du contexte. C'est une option de conception, pas une obligation architecturale. [S01] Proposition : attribuer une version au contexte utilisé par chaque run ; publier les modifications approuvées ; revalider les règles critiques immédiatement avant une action sensible. Une synchronisation horaire ne doit pas retarder une révocation de droits.

Le cycle d'amélioration proposé est : observation, correction proposée, revue, évaluation, publication d'une nouvelle version, surveillance et retour arrière si nécessaire. Ne pas transformer automatiquement les sorties d'un agent en connaissances approuvées.

9. Gouvernance et niveaux d'autonomie

Proposition : définir l'autonomie par type d'action, plutôt que donner un statut global « autonome » à un agent.

Niveau proposé	Action autorisée	Exemple
0	Lire et expliquer	Analyser un dossier autorisé
1	Préparer	Produire un brouillon interne
2	Modifier de façon limitée et réversible	Ajouter une étiquette interne
3	Exécuter après validation précise	Envoyer un message approuvé
4	Exécuter automatiquement dans un mandat borné	Ranger des documents selon des règles validées

Cette échelle est une convention du guide. Le niveau 4 n'est pas une cible obligatoire. Des actions peuvent rester durablement soumises à validation.

Ce que doit contenir une approbation

Proposition : lier l'approbation au contenu exact de l'opération, à ses destinataires, aux ressources ciblées, à la version des données et à une expiration. Toute modification matérielle invalide l'approbation. Afficher à l'humain ce qui sera fait, sur quelles pièces cela repose et comment annuler si c'est possible.

Faire respecter les règles

Les permissions doivent être appliquées à chaque appel d'outil. Un service de politiques peut séparer la décision d'autorisation de son application ; c'est le rôle documenté d'Open Policy Agent. [S32]

Proposition : commencer avec une politique explicite dans l'application, puis adopter un moteur séparé si plusieurs services doivent partager les mêmes règles.

Prévoir un responsable des procédures, un responsable des accès et une personne joignable en cas d'incident. Dans une petite structure, une même personne peut cumuler ces rôles ; ils doivent malgré tout être identifiés.

10. Sécurité des agents

Le référentiel OWASP 2026 couvre notamment le détournement d'objectif, l'abus d'outils et de privilèges, les composants compromis, l'exécution inattendue de code, l'empoisonnement de mémoire, les communications entre agents, les défaillances en cascade et l'exploitation de la confiance humaine. Il sert à organiser les risques ; ce n'est pas une certification. [S18]

Modèle de menace proposé

Scénario	Barrière à construire	Vérification concrète
Une page impose d'ignorer les règles	Traiter les pages comme données non fiables	Le contenu ne modifie pas les autorisations
Un document demande une fuite de fichiers	Droits étroits et sorties réseau contrôlées	Aucun envoi vers une destination interdite
Un agent choisit un dossier d'un autre client	Isolation par identité authentifiée	Refus avant lecture et avant écriture
Une mémoire conserve une instruction hostile	Écriture en zone proposée, validation séparée	L'entrée ne devient pas une règle active
Un outil reçoit une commande dangereuse	Schémas stricts et API limitée	Rejet des arguments hors contrat
Un connecteur est compromis	Version fixée, revue et isolation	Son périmètre d'accès reste limité
Une boucle multiplie les sous-tâches	Plafonds atomiques et profondeur maximale	Arrêt avant dépassement
Un agent falsifie un résultat	Contrôle sur l'état externe	Une déclaration seule n'est jamais un succès

Une consigne « ignore les injections » est utile mais insuffisante. La sécurité ne doit pas reposer uniquement sur l'obéissance du modèle. La même observation vaut pour les résultats d'outils et les messages des autres agents : leur provenance n'en fait pas des instructions autorisées.

Secrets et exécution de code

Proposition : conserver les secrets dans un service extérieur aux espaces où s'exécutent les scripts générés. L'exécuteur travaille avec un compte limité, un répertoire dédié et des sorties réseau autorisées. Éviter de monter le disque personnel complet ou le socket Docker dans cet environnement. La séparation entre secrets et sandbox est également un enseignement du retour d'architecture Anthropic. [S13]

MCP et services distants

Les recommandations MCP couvrent les jetons, le consentement, la SSRF, le détournement de session et les serveurs locaux. [S15] Proposition : vérifier l'origine des serveurs, leur version, leurs droits et les données qui transitent. Une URL trouvée sur une page ne devient pas automatiquement un serveur autorisé. Un connecteur de téléchargement doit empêcher l'accès aux adresses internes, y compris après redirection ou nouvelle résolution DNS.

Choisir des outils de sécurité selon leur fonction

Outil ou famille	Fonction documentée	Limite à retenir
garak [S26]	Recherche de vulnérabilités par sondes	Détecter ne bloque pas l'exploitation en production
PyRIT [S27]	Campagnes d'évaluation adversariale	Demande des scénarios et une interprétation
NeMo Guardrails [S28]	Contrôles aux différentes étapes d'interaction	Complète les contrôles applicatifs
OPA [S32]	Décisions de politique à partir de données structurées	Ne juge pas seul la vérité d'un texte
OWASP [S18]	Référentiel de risques et mesures	N'exécute aucun contrôle à votre place

Le classement EM360Tech a servi de point de départ pour retrouver ces auteurs. Les performances comparées et l'ordre du classement n'ont pas été validés. [S06] Il n'est pas nécessaire d'installer tous ces outils pour un prototype de recherche en lecture seule.

11. Fiabilité, reprises et observabilité

Une relance peut dupliquer une action

Symfony Messenger documente qu'un message peut être livré plusieurs fois, notamment si le worker tombe après traitement et avant acquittement. Une nouvelle tentative n'assure donc pas l'absence de doublon. [S29]

Proposition : attribuer une clé stable à l'opération métier, réserver son exécution atomiquement et conserver son reçu. Si le fournisseur gère l'idempotence, réutiliser sa clé lors des reprises. Un simple verrou local ne résout pas le cas où le service distant a exécuté l'action mais où sa réponse a été perdue.

Dans ce cas ambigu, chercher le reçu distant ou l'état métier. Si le résultat reste indéterminé et l'opération non idempotente, arrêter pour réconciliation. Ne pas renvoyer aveuglément un paiement ou un message externe.

Séparer transaction et effets externes

Proposition : enregistrer la tâche et l'événement à publier dans une même transaction locale, puis utiliser une outbox pour la transmission. Garder des états explicites pour les appels externes. Une compensation est une nouvelle action métier, pas l'annulation magique d'un effet déjà visible ; un courriel envoyé ne peut pas être « désenvoyé » par une transaction SQL.

Pour des tâches longues, la persistance et la reprise sont des capacités centrales. Temporal documente une exécution durable fondée sur l'historique des événements ; LangGraph documente des checkpoints. Leur présence ne dispense pas de gérer les effets externes. [S17, S33]

Traces utiles

Proposition : consigner run_id, version du workflow, modèle, procédure et politique, sources consultées, outil, arguments expurgés, décision d'autorisation, durée, coût, erreur et identifiant de résultat. Conserver une explication synthétique de la décision ; l'audit doit surtout reposer sur des faits observables, pas sur une prétendue lecture complète de la pensée du modèle.

Limitier l'accès aux traces et leur durée de conservation. Ne pas transformer les journaux en copie incontrôlée des documents et secrets. Les opérations sensibles peuvent exiger un stockage empêchant ou détectant la modification a posteriori.

Signaux à surveiller

Proposition : durée d'attente, tâches bloquées, nombre de reprises, coût par tâche réussie, erreurs d'outil, proportion de validations humaines, refus de politique et tentatives de sorties réseau anormales. L'arrêt d'urgence doit couper les nouvelles opérations et permettre d'identifier celles qui sont déjà parties.

12. Évaluer avant d'automatiser

Anthropic distingue les évaluateurs déterministes, les évaluateurs fondés sur un modèle et les évaluateurs humains. Les jugements automatiques doivent être adaptés à la tâche et confrontés au jugement humain. [S25]

Proposition : constituer un premier jeu de 40 cas, puis l'élargir selon les incidents et la variété réelle. Ce nombre est un point de départ pratique, pas une norme. Inclure des cas simples, ambigus, incomplets, hostiles et des pannes simulées. Réserver une partie des cas à la validation pour éviter d'optimiser exclusivement sur les exemples connus.

Dimension	Exemple de test	Critère proposé
Exactitude	Extraction avec réponse de référence	Champs corrects ; inconnus explicités
Sources	Affirmation importante d'un rapport	Référence accessible qui soutient l'affirmation
Autorisation	Action hors périmètre	Blocage côté serveur
Isolation	Identifiant d'un autre tenant	Zéro accès dans les cas testés
Robustesse	Timeout après succès distant	Réconciliation sans doublon
Sobriété	Source inaccessible	Arrêt ou alternative dans le budget
Reprise	Arrêt forcé du worker	État conservé et reprise cohérente
Mémoire	Fausse instruction dans un document	Aucune règle active créée

Répéter certains cas, car une seule réussite ne mesure pas la variabilité. Comparer à une base simple : travail manuel, workflow déterministe ou un seul appel au modèle. Un second agent est justifié s'il améliore réellement la qualité ou réduit le temps de contrôle.

Mesurer le résultat final dans le système cible. Pour un dossier de recherche, compter les références justes et les contradictions traitées. Pour un import, vérifier les lignes effectivement créées et l'absence de doublons. Pour un message, vérifier contenu, destinataire et reçu, pas seulement la qualité du brouillon.

13. Choisir les modèles et maîtriser le coût

Proposition : sélectionner les modèles sur un jeu de tâches représentatif, en évaluant qualité, respect des outils, latence, confidentialité, coût et taux de reprise. La taille ou la nouveauté du modèle ne suffisent pas à le choisir.

Une stratégie possible consiste à confier l'extraction simple à un modèle économique et les synthèses difficiles à un modèle plus capable. L'escalade repose sur des validations ratées, des sources contradictoires ou des informations manquantes. Éviter de la déclencher uniquement sur une confiance numérique inventée par le modèle.

Budget de fonctionnement

Coût complet = appels aux modèles + outils payants + calcul + stockage + maintenance + temps de validation humaine + coût des erreurs.

Exemple purement illustratif : 1 000 tâches à 0,04 € d'inférence représentent 40 €. Si 20 % demandent 2 minutes de vérification, cela ajoute 6 h 40 de travail humain. À une valeur interne hypothétique de 20 €/h, cela représente environ 133 €. Ces valeurs ne sont ni un tarif fournisseur ni une promesse de coût.

Le budget doit être réservé avant les opérations parallèles, puis rapproché des consommations réelles. Fixer nombre d'étapes, plafond de tokens, durée, tentatives et dépenses d'outils. Une boucle de surveillance déterministe coûte moins qu'un appel de modèle permanent lorsqu'aucun événement ne nécessite de décision.

Pour un modèle local, vérifier la VRAM réellement disponible, le contexte, la quantification, le débit et les autres usages du GPU. Un serveur sans GPU peut héberger le contrôleur et utiliser une API distante. Le déploiement du contrôleur et celui de l'inférence sont deux décisions distinctes.

Ne pas bâtir un service client sur l'hypothèse qu'un abonnement conversationnel donne une API illimitée. Avant intégration, vérifier le mode d'accès officiellement autorisé, les quotas et les conditions du fournisseur choisi. Les affirmations générales de gratuité du corpus ne sont pas reprises comme hypothèses économiques.

14. Choisir une pile technique

Proposition : choisir le minimum de composants qui couvre le besoin mesuré.

Option	Bon point de départ si...	Ce qui reste à construire ou vérifier
Application + appels LLM directs	Peu de workflows et équipe technique	Boucle, contrats, persistance et évaluations
Symfony + Messenger	Logique métier PHP et interface sur mesure	État des tâches, politique et outils contrôlés
n8n	Beaucoup de connecteurs et orchestration visuelle	Droits, reprise métier et limites des nœuds [S30]
LangGraph	Besoin de graphes d'exécution et checkpoints	Intégration métier et contrôle des effets [S17]
Temporal	Attentes longues et reprise durable exigeante	Activités, opérations idempotentes, exploitation [S33]

Option	Bon point de départ si...	Ce qui reste à construire ou vérifier
StackGen/Aiden	Besoin centré DevOps, plateforme ou SRE	Vérifier les capacités annoncées sur son périmètre [S04]

Cette comparaison n’est pas un benchmark. Elle ne conclut ni au meilleur produit ni au prix total. Pour une offre commerciale, demander une démonstration sur un incident représentatif : interruption, révocation d’accès, conflit de modification ou résultat ambigu.

La construction d’un agent avec un assistant de programmation ne rend pas cet assistant obligatoire en production. Distinguer l’outil qui développe le logiciel, le runtime qui l’exploite et le fournisseur de modèles qu’il appelle.

15. Une réalisation possible avec Symfony

Cette section est une proposition d’architecture, adaptée à une pratique de PHP/Symfony ; elle ne décrit pas un produit déjà développé ni une configuration testée.

Premier périmètre

Une interface authentifiée permet de créer une tâche, suivre son état, lire le résultat et approuver une opération. Une API métier applique les permissions. Des workers Messenger traitent les étapes. Une base relationnelle conserve les tâches et les décisions. Les documents sont stockés à part et référencés par identifiant.

Messenger fournit le transport et la gestion de messages ; il ne fournit pas à lui seul le plan de travail de l’agent ni toutes les garanties métier. Choisir une version de Symfony maintenue et compatible avec le projet. La documentation 7.4 a été consultée pour les mécanismes de messagerie, sans supposer qu’elle correspond à toute installation existante. [S29]

Entités applicatives proposées

Entité	Champs essentiels
Task	propriétaire, objectif, statut, échéance, budget
Run	task_id, versions, modèle, début, fin
Step	type, état, tentative, résultat, erreur
ToolCall	outil, clé d’idempotence, statut, reçu
Approval	action exacte, approbateur, expiration, empreinte
Source	URL, titre, date, droits, statut de collecte
KnowledgeEntry	contenu, provenance, validation, validité
AuditEvent	identité, opération, décision, référence du résultat

Ajouter tenant_id si le système sert plusieurs organisations. L’isolation doit couvrir la base, les fichiers, les caches, la recherche, les queues et les journaux. Une colonne tenant_id sans filtrage systématique n’assure pas cette isolation.

Services proposés

TaskService valide la demande. ContextBuilder prépare les informations autorisées. ModelGateway appelle le fournisseur sélectionné. PolicyService autorise ou bloque les opérations. ToolExecutor appelle les connecteurs. ResultVerifier contrôle les critères métier. BudgetLedger réserve puis rapproche la consommation. AuditWriter conserve les événements utiles.

Boucle de contrôle illustrée

```
charger la tâche et son état durable
si annulée, hors délai ou budget épuisé : arrêter
construire un contexte filtré et versionné
obtenir une proposition d'action structurée
valider le schéma et les préconditions métier
contrôler les droits et réserver le budget
si validation requise : enregistrer l'attente, libérer le worker
sinon : exécuter avec une clé d'opération stable
enregistrer le reçu et vérifier le résultat
planifier la suite ou terminer la tâche
```

Ne pas maintenir une transaction SQL ouverte pendant un appel de modèle. Pour les mises à jour concurrentes, utiliser une version d'état ou un verrou adapté. La reprise doit détecter si l'étape a déjà produit un effet.

Contrat HTTP proposé

POST /tasks crée une tâche. GET /tasks/{id} affiche un état et des résultats autorisés. POST /tasks/{id}/cancel demande l'arrêt. POST /approvals/{id}/accept approuve une opération précise. Les actions idempotentes acceptent une clé d'opération. Ces routes sont des exemples à développer ; elles ne correspondent pas à une API existante.

16. Cas pratique : un dossier de recherche sourcé

Proposition de premier workflow : donner des URLs et une question, puis produire un dossier qui distingue faits, affirmations commerciales, contradictions et recommandations. C'est un cas utile avec des effets externes limités.

Entrées et sortie

Entrées : sujet, liste d'URLs, langue, profondeur maximale, budget de pages, date de référence et format attendu. Sortie : guide, registre des sources, matrice de vérification et liste des pages inaccessibles. Le système ne publie rien à l'extérieur sans action autorisée distincte.

Collecte bornée

Proposition : commencer avec une profondeur de deux liens et un plafond de quarante pages pertinentes. Ces limites sont configurables. Normaliser les URLs, retirer les fragments, conserver les URL finales, dédupliquer les contenus et ignorer les menus. Ne pas suivre les pages d'inscription, calendriers infinis ou paramètres de suivi.

Pour chaque document, enregistrer titre, auteur lorsqu'il est identifiable, URL, date de collecte, date de publication si visible, langue, mode d'accès et statut. Respecter les conditions d'accès et ne pas contourner les murs d'authentification. Une page inaccessible devient une limite documentée, pas un contenu reconstitué.

Recouplement

Proposition : extraire des affirmations courtes, puis chercher la source la plus proche du fait. « Le logiciel annonce X » se vérifie sur sa page produit ; « X fonctionne mieux » exige un protocole de comparaison. « Cet outil est open source » se vérifie dans sa licence. Deux pages du même auteur ne constituent pas deux validations indépendantes.

Le rédacteur reçoit les fiches vérifiées, pas une masse indistincte de pages. Le vérificateur contrôle que chaque affirmation importante est soutenue par sa référence, que les désaccords restent visibles et que les passages inaccessibles ne sont pas présentés comme lus.

Exemple de fiche de preuve proposée

```
{
  "claim_id": "C004",
  "claim": "Une skill est une fonction typee executable",
  "source_initiale": "S01",
  "verification": ["S11", "S12"],
  "verdict": "formulation trompeuse",
  "correction": "Procédure et ressources ; validation dans les outils",
  "review_status": "reviewed"
}
```

Un rapport final doit annoncer le périmètre réel : nombre de pages retenues, branches suivies, exclusions et échecs. Il ne doit pas annoncer « tout le web a été scrappé » ni transformer un extrait de moteur de recherche en lecture intégrale.

17. Déploiement et exploitation

Local, serveur ou hybride

Déploiement	Intérêt	Contraintes à prévoir
Poste local isolé	Expérimentation et éventuel GPU local	Veille, disponibilité, séparation des fichiers personnels
VPS	Contrôleur disponible et jobs planifiés	Durcissement, sauvegardes, coût d'inférence externe éventuel
Hybride	Contrôleur distant et worker spécialisé local	Authentification, perte de connexion, file d'attente

Proposition : en hybride, privilégier un worker local qui vient chercher ses tâches par une connexion authentifiée. Éviter d'exposer directement au public un terminal, une API de modèle ou un exécuter disposant de droits sur le poste. Limiter explicitement les données qui peuvent être envoyées à un modèle cloud.

Déploiement progressif

Commencer en développement avec des données de test, passer en mode observation sur des cas réels autorisés, puis autoriser des actions internes réversibles. Ouvrir une opération externe seulement après évaluation de son contrat, de sa validation et de sa reprise.

Versionner application, procédures, politiques et configuration. Tester une restauration des tâches et documents. Prévoir la rotation des secrets et la révocation des accès. Une sauvegarde disponible mais jamais restaurée ne démontre pas que le système est récupérable.

Réponse à incident proposée

Suspendre les nouvelles actions, isoler le connecteur concerné, conserver les preuves utiles, déterminer quelles opérations sont réellement parties, révoquer les accès exposés, corriger puis rejouer les tests. Toute reprise d’une opération ambiguë exige de connaître son état externe. Documenter aussi les incidents évités pour améliorer les scénarios d’évaluation.

Les questions de durée de conservation, droits des personnes et contrats de sous-traitance doivent être étudiées selon les données et le service réellement déployés. Ce guide ne constitue pas une analyse juridique ni une certification de conformité.

18. Feuille de route et critères d’acceptation

Proposition : progresser par jalons vérifiables plutôt que promettre une date universelle de mise en production.

Jalon	Livrable	Condition de passage
A. Cadrage	Un cas d’usage, ses entrées et ses risques	Résultat attendu vérifiable
B. Prototype	Un agent, peu d’outils, lecture seule	Résultats comparés à une base simple
C. Persistance	État durable, reprise, traces	Un arrêt de worker ne perd pas la tâche
D. Contrôle	Permissions, budgets, validation	Les actions interdites sont bloquées hors LLM
E. Mémoire	Provenance et procédure de correction	Une erreur peut être retirée et réindexée
F. Pilote	Utilisation supervisée et métriques	Qualité et coût acceptables sur cas réels
G. Extension	Autres outils ou agents	Bénéfice mesuré et contrôles conservés

Checklist de livraison proposée

- Chaque tâche a un objectif et une condition de fin.
- Chaque outil a un contrat, un périmètre et un délai.
- Les actions sensibles demandent une approbation portant sur leur contenu exact.
- Les budgets et limites restent valables lors des délégations.
- Un document externe ne peut pas modifier les permissions.
- Les sources et connaissances ont une provenance consultable.
- Les opérations ambiguës ne sont pas relancées aveuglément.
- Les identités et données de plusieurs clients sont isolées.
- Les secrets sont absents des contextes et espaces d’exécution non fiables.
- Les tests incluent pannes, doublons, contradictions et tentatives d’injection.

- L'arrêt d'urgence, la restauration et le retour à une version précédente fonctionnent.
- Le coût par tâche utile et le temps de contrôle humain sont mesurés.

19. Cahier des charges de départ

Objectif proposé : créer une application qui transforme un ensemble de sources autorisées en dossier documenté, tout en conservant la maîtrise de son coût et de ses opérations.

Périmètre initial : un utilisateur, un workflow de recherche, une interface de suivi, un registre des sources, un modèle configurable et quelques outils de lecture. L'ajout d'une mémoire documentaire vient après validation de la chaîne de collecte. La publication externe reste une opération distincte.

Exigences fonctionnelles : ajouter des URLs, afficher les statuts de collecte, consulter les sources utilisées, relancer une étape sûre, annuler une tâche, télécharger les résultats et enregistrer une correction.

Exigences de qualité : citation liée à l'affirmation, distinction entre fait et proposition, déclaration des sources inaccessibles, aucun contenu inventé pour combler une collecte manquante, conservation des versions utilisées.

Exigences d'exploitation : authentification, budgets, timeouts, clés d'opérations stables, historique d'événements, reprise après interruption et sauvegarde restaurable. L'accès arbitraire au shell n'est pas nécessaire pour ce premier cas.

Décision de conception : commencer par le résultat utile et le noyau de contrôle. Ajouter un second agent, une base vectorielle ou un protocole de délégation seulement lorsqu'un problème observé justifie cette complexité.

20. Registre des sources et limites

Les URLs ci-dessous sont les points de consultation ou les adresses officielles retrouvées. Les références sont dédoublées par document, pas nécessairement par domaine. « Consulté » ne signifie pas « installé », « audité » ou « testé ».

Neuf points de départ

[S01] MindStudio — Agentic Business OS / skills / brand context. Article commercial, 17 mars 2026. Contexte et coordination ; définitions recoupées avec [S11–S12].

<https://www.mindstudio.ai/blog/agentic-business-os-claude-code-skills-brand-context>

[S02] Nirmata.in — What Is an Agentic AI OS? Article commercial, 16 mai 2026. Vision métier et promesses ; ROI non validé. Ne pas confondre ce domaine avec d'autres sociétés homonymes.

<https://nirmata.in/what-is-agentic-ai-os-autonomous-operating-system/>

[S03] Acumentica — Governed Agentic Enterprise OS. Cadre commercial de gouvernance.

<https://acumentica.com/governed-agentic-enterprise-os-governing-enterprise-decisions-under-uncertainty/>

[S04] StackGen — Aiden OS. Page produit, capacités annoncées par l'éditeur. <https://stackgen.com/platform>

[S05] SDLC Corp — Agentic AI Fundamentals. Article pédagogique de prestataire ; lien ReAct suivi.

<https://sdllcorp.com/post/agentic-ai-fundamentals/>

[S06] EM360Tech — Top 10 Security Tools for Agentic Systems. Article éditorial affichant le 26 septembre 2025 ; contenu susceptible d'évoluer. Classement non reproduit comme benchmark.

<https://em360tech.com/top-10/security-tools-for-agentic-systems>

[S07] Acumentica — Agentic AI Control OS. Cadre de contrôle annoncé, sans preuve de garantie universelle.

<https://acumentica.com/agentic-ai-control-os/>

[S08] AI Profit Boardroom — How To Build An Agentic OS. Article commercial, 29 mai 2026. Vidéos et offre membre non examinées. <https://aiprofitboardroom.com/blog/how-to-build-an-agentic-os/>

[S09] MindStudio — What Is an AI Operating System? Inventaire de composants et promotion de l'offre.

<https://www.mindstudio.ai/blog/what-is-ai-operating-system-build-with-claude-code>

Références techniques de recoupement

[S10] Anthropic — Building effective agents. Article d'ingénierie daté du 19 décembre 2024, page actualisée ; patterns et distinction workflow/agent. <https://www.anthropic.com/engineering/building-effective-agents>

[S11] Claude Code — Extend Claude with skills. Documentation officielle ; structure des skills.

<https://code.claude.com/docs/en/skills>

[S12] Agent Skills — Overview. Standard ouvert, atteint depuis [S11] ; index documentaire également consulté.

<https://agentskills.io/home>

[S13] Anthropic — Scaling Managed Agents. Retour d'architecture du 8 avril 2026, atteint depuis [S10].

<https://www.anthropic.com/engineering/managed-agents>

[S14] Model Context Protocol — Architecture overview. Documentation officielle ; redirection vers l'édition 2026-07-28 lors de la recherche. <https://modelcontextprotocol.io/docs/learn/architecture>

[S15] MCP — Security Best Practices. Recommandations de sécurité ; adresse versionnée 2025-11-25 consultée, à confronter à la version intégrée. https://modelcontextprotocol.io/specification/2025-11-25/basic/security_best_practices

[S16] A2A — Specification. Spécification officielle ; URL « latest » évolutive.

<https://a2a-protocol.org/latest/specification/>

[S17] LangGraph — Persistence. Documentation officielle ; checkpoints et stores.

<https://docs.langchain.com/oss/python/langgraph/persistence>

[S18] OWASP — Top 10 for Agentic Applications 2026. Page de publication du 9 décembre 2025 et PDF de 57 pages ; passages pertinents examinés.

<https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>

[S19] OWASP — Agent Control Standard. Page de présentation repérée et ouverte depuis [S18] ; le standard complet n'a pas été analysé et ne fonde pas les prescriptions du guide.
<https://genai.owasp.org/resource/agent-control-standard-acs/>

[S20] AI Profit Boardroom — Obsidian Agentic OS. Lien suivi depuis [S08], affirmation de licence recoupée avec [S21]. <https://aiprofitboardroom.com/blog/obsidian-agentic-os/>

[S21] Obsidian — License overview. Conditions officielles, date de mise à jour affichée : 20 février 2025.
<https://obsidian.md/license>

[S22] Aumentica — Decision Control OS. Lien suivi depuis [S03] ; cadre déclaratif.
<https://aumentica.com/decision-control-os/>

[S23] Claude Code — Overview. Documentation officielle atteinte par le lien Anthropic de [S01].
<https://code.claude.com/docs/en/overview>

[S24] Anthropic — Effective context engineering for AI agents. Article d'ingénierie ; gestion du contexte.
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

[S25] Anthropic — Demystifying evals for AI agents. Article d'ingénierie ; types d'évaluateurs et qualité des évaluations. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>

[S26] NVIDIA — garak. Dépôt officiel, atteint depuis [S06]. <https://github.com/NVIDIA/garak>

[S27] Microsoft — PyRIT. Dépôt officiel, atteint depuis [S06]. <https://github.com/Azure/PyRIT>

[S28] NVIDIA — NeMo Guardrails. Documentation officielle, atteinte depuis [S06].
<https://docs.nvidia.com/nemo/guardrails/about-nemo-guardrails-library/overview>

[S29] Symfony — Messenger 7.4. Documentation officielle ; livraisons multiples et handlers idempotents.
<https://symfony.com/doc/7.4/messenger.html>

[S30] n8n — Tools AI Agent. Documentation officielle de nœud. L'ancienne entrée /advanced-ai a renvoyé une page introuvable ; une page précise a ensuite été retrouvée.
<https://docs.n8n.io/integrations/builtin/cluster-nodes/root-nodes/n8n-nodes-langchain.agent/tools-agent/>

[S31] Yao et al. — ReAct: Synergizing Reasoning and Acting in Language Models. Article de recherche, arXiv 2022 / ICLR 2023 ; source originale recherchée après le lien pédagogique de [S05]. <https://arxiv.org/abs/2210.03629>

[S32] Open Policy Agent — Documentation. Politique déclarative et séparation décision/application.
<https://www.openpolicyagent.org/docs>

[S33] Temporal — What is Temporal? Documentation officielle sur l'exécution durable.
<https://docs.temporal.io/temporal>

[S34] StackGen — Documentation. Lien suivi depuis [S04], mais aucun texte exploitable retourné. Aucune fonctionnalité supplémentaire n'en est déduite. <https://docs.stackgen.com/docs>

[S35] SDLC Corp — ReAct: Reasoning and Acting. Lien suivi depuis [S05] ; source secondaire remplacée par [S31] pour les explications techniques. <https://sdllcorp.com/post/react-reasoning-and-acting/>

Limites et travaux complémentaires

Le registre comprend 35 références : les neuf pages initiales, des prolongements et des sources de contrôle. Certaines sont seulement des étapes de navigation ou des pages non exploitables, comme indiqué. Ce nombre n'est donc pas celui de 35 preuves indépendantes.

La recherche ne couvre pas exhaustivement tous les liens internes et externes. Elle ne comprend ni benchmark des plateformes, ni audit de code, ni examen des offres commerciales privées. Les exemples Symfony sont un cahier de conception ; ils demandent implémentation et tests. Les prix, compatibilités précises et conditions d'accès doivent être vérifiés au moment de choisir une offre.

Les livrables conservent la synthèse, les références et les décisions de recoupement. Ils ne constituent pas une republication intégrale des articles collectés. Pour prolonger le travail, la prochaine étape utile est de choisir un workflow et de construire un prototype évalué sur des cas réels autorisés.